

ApplyNow

Admissions Portal

Full Project Documentation

Project Type	Intermediate Frontend Development Final Project
Developer	Albashir Maigari Beti
Frontend	React, Tailwind CSS, React Router
API / Data	Axios, JSON Server
Deployment	Netlify (frontend), Render (API)

A responsive admissions application that supports multi-step application submission, status tracking, administrative review, and a rule-based admission likelihood predictor.

1. Project Overview

ApplyNow is a responsive web-based admissions portal developed as the final project for the Intermediate Frontend Development pathway. The system demonstrates a complete frontend workflow for collecting applicant information, submitting applications, tracking application status, reviewing applications from an administrative interface, and presenting a rule-based admission prediction.

The project was designed around a Nigerian admissions context. Academic information includes WAEC, NECO and NABTEB examination types, nine O'Level subject-and-grade entries, and a JAMB/UTME score.

2. Problem Statement and Pain Points

Admission processes can become difficult for applicants when forms are long, status information is not easy to access, and feedback is delayed. Administrators also need a clear way to review submitted applications and record decisions.

- Long or complicated admission application processes.
- Difficulty for applicants in tracking the status of a submitted application.
- Lack of immediate feedback after application submission.
- Inefficient review and management of multiple applications.
- Need for a simple way to estimate admission likelihood from academic results.
- Need for a responsive interface that works on both desktop and mobile devices.

3. Aim and Objectives

The aim of ApplyNow is to provide a simple, responsive and functional admissions portal that demonstrates the major concepts learned during Intermediate Frontend Development.

- Build a multi-step admission form for personal, academic and guardian information.
- Validate user input before allowing progression or submission.
- Generate a unique Application ID for tracking.
- Provide an applicant dashboard for retrieving submitted applications and viewing status.
- Provide an admin dashboard for reviewing, approving and rejecting applications.
- Synchronize admin decisions with the applicant-facing dashboard through a shared API.
- Implement a rule-based admission status predictor using JAMB and O'Level performance.
- Deploy the frontend and mock API so the project can be demonstrated online.

4. Core Features

Module	Main Function
Application Form	Four-step form covering personal, academic, guardian, and review/submit stages.
Validation	Checks required fields, NIN format, JAMB range, O'Level entries and duplicate subjects.
State/LGA	Dependent Nigerian State and LGA dropdowns; changing state resets the selected LGA.

Module	Main Function
Confirmation	Displays successful submission information and the generated Application ID.
Applicant Dashboard	Searches by Application ID and displays applicant details, results and status.
Status Predictor	Calculates a rule-based likelihood score from JAMB and O'Level performance.
Admin Dashboard	Lists applications, statistics, pending reviews, and approve/reject actions.
Responsive UI	Layouts, forms, tables and navigation adapt to smaller screens.

5. Application Workflow

The main end-to-end workflow of the system is:

- The applicant opens ApplyNow and selects Apply Now.
- The applicant completes Personal Information.
- The applicant completes Academic Information, including examination details, nine O'Level subjects and JAMB score.
- The applicant completes Guardian Information.
- The Review & Submit stage displays the entered information for confirmation.
- The application is posted to the JSON Server API through Axios.
- The system generates an Application ID and stores the application with an initial status of Under Review.
- The applicant searches the Application ID from the Dashboard.
- The admin reviews the application and changes its status to Approved or Rejected.
- When the applicant searches again, the Dashboard retrieves and displays the updated status.

6. Multi-Step Application Form

6.1 Personal Information

The personal information stage collects the applicant's first name, last name, email, phone number, date of birth, NIN, state, LGA and residential address. NIN validation requires an 11-digit value.

6.2 Academic Information

The academic stage is adapted to the Nigerian admissions context. Applicants select an examination type such as WAEC, NECO or NABTEB, provide the examination year, secondary school, intended course and JAMB score. The JAMB score is validated within the 0-400 range.

Nine O'Level subject-and-grade pairs are provided. Grade options range from A1 to F9, and duplicate subject selection is prevented.

6.3 Guardian Information

The guardian stage collects the guardian's full name, relationship, phone number, email and address.

6.4 Review and Submit

Before submission, the applicant can review personal, academic and guardian information. O'Level results are displayed in a table so the applicant can verify the selected subjects and grades.

7. Applicant Dashboard

The Applicant Dashboard allows an applicant to retrieve an application using the generated Application ID. The dashboard displays application details, applicant information, O'Level results, the current status, and the admission status predictor.

The dashboard supports the tracking requirement without requiring the applicant to re-enter the full application.

8. Admin Dashboard

The Admin Dashboard provides an overview of submitted applications. It displays totals for applications that are Under Review, Approved and Rejected. Pending applications are prioritized for review, while decided applications remain available in the complete list.

The admin can inspect applicant details and update an application to Approved or Rejected. The update is sent to the same API used by the applicant dashboard, which keeps both views synchronized.

9. Rule-Based Admission Status Predictor

The project implements the rule-based option for the Admission Status Predictor. It is not a trained machine-learning model. Instead, predefined scoring rules convert academic performance into a simple likelihood score.

JAMB performance is normalized from the 0-400 JAMB scale to a percentage. O'Level grades are mapped to numerical scores and averaged. The final prediction score uses the following weighting:

Component	Weight
JAMB performance	60%
O'Level performance	40%

The combined score is classified as High Likelihood, Moderate Likelihood or Low Likelihood. The predictor is included for demonstration purposes only and does not represent an official admission decision.

10. Technologies Used

Technology	Purpose
React	Component-based frontend development and state-driven user interfaces.
JavaScript	Application logic, validation, calculations and event handling.
Tailwind CSS	Responsive styling and interface layout.
React Router	Client-side navigation between Home, Apply, Dashboard, Admin and Confirmation pages.
Axios	HTTP requests between the React frontend and the API.
JSON Server	Mock REST API for storing and updating application records.
Vite	Development server and production build tooling.
Git / GitHub	Version control and source-code hosting.
Netlify	Frontend deployment.
Render	Hosted JSON Server API.

11. Project Structure

```
applynow-admissions-portal/  
  public/  
    _redirects  
    icons.svg  
  src/  
    assets/  
    components/  
      Hero.jsx  
      Navbar.jsx  
      ProcessSection.jsx  
    data/  
      nigeriaLocations.js  
    pages/  
      Admin.jsx  
      ApplicationForm.jsx  
      Confirmation.jsx  
      Dashboard.jsx  
    services/  
      api.js  
    App.jsx  
    index.css  
    main.jsx  
  db.json  
  README.md  
  package.json  
  vite.config.js
```

12. API Integration and Data Handling

Axios is configured in a reusable API service. During local development, the application can communicate with JSON Server on localhost. In production, the frontend uses a Vite environment variable (VITE_API_URL) to connect to the hosted Render API.

The application uses REST-style operations to create application records, retrieve applications for the applicant and admin dashboards, and update application status.

13. Responsive Design

The interface was tested for desktop and mobile use. Tailwind CSS responsive utilities are used for layouts, navigation, forms, cards and tables. The navigation includes a functional mobile menu, and the multi-step form remains usable on narrow screens.

14. Deployment

The project uses separate deployments for the frontend and API. The React application is hosted on Netlify, while the JSON Server API is hosted on Render. A Netlify redirect rule supports React Router routes when pages are opened or refreshed directly.

Resource	URL
Live Frontend	https://applynow-admissions-portal.netlify.app/
Live API	https://applynow-admissions-api.onrender.com

15. Challenges and Solutions

Challenge	Solution
Connecting the complete applicant and administrator to the mock JSON Server API so submission, retrieval and status updates operate on the same shared database.	Used a shared JSON Server API so submission, retrieval and status updates operate on the same shared database.
Dependent State/LGA selection	Created a Nigerian locations data module and reset the LGA whenever the state changes.
Keeping long forms manageable	Split the application into four steps with progress indicators and per-step validation.
Tracking applications	Generated a unique Application ID and implemented dashboard search by that identifier.
Deploying frontend and API separately	Used an environment variable for the API base URL and deployed React to Netlify and JSON Server to Heroku.

16. Skills and Learning Outcomes

- Building structured applications with React and reusable components.
- Managing form data, errors, loading states and UI changes with React state.
- Implementing multi-step forms and client-side validation.
- Using React Router for page navigation.
- Using Axios to perform API requests.
- Using JSON Server to simulate REST API operations.
- Creating responsive interfaces with Tailwind CSS.
- Implementing dependent dropdowns and rule-based scoring logic.
- Using Git and GitHub for version control.
- Preparing a Vite application for production and deploying frontend and API services.

17. Limitations and Future Improvements

ApplyNow is an educational frontend project and uses a mock JSON Server API. It does not currently implement production authentication or authorization. For that reason, the deployed demonstration should only use fictional or test applicant information.

- Add secure applicant and administrator authentication.
- Use a production database and backend API with proper access control.
- Add role-based authorization for administrative actions.
- Add email or notification support for application decisions.
- Allow document uploads such as result slips or certificates.
- Add stronger reporting and filtering tools for administrators.
- Replace or extend the rule-based predictor with a validated model if a future project requires machine learning.

18. Conclusion

ApplyNow demonstrates an end-to-end admissions workflow built with the technologies covered in the Intermediate Frontend Development pathway. The project combines React components, state management, validation, routing, responsive design, API integration, version control and

deployment in one working application.

Applicants can submit and track applications, while administrators can review and update decisions. The rule-based predictor provides an additional demonstration of how academic data can be transformed into a simple likelihood score.

19. Security and Demonstration Note

The public deployment is intended for demonstration and educational use. Because the current mock API does not include production-grade authentication and access control, real NINs, phone numbers, email addresses, residential addresses or other sensitive personal information should not be entered into the deployed application.